Yes, we can make HiveNode containers handle text, voice, and (to a limited degree) video over pure mesh — without relying on ISP cellular or traditional internet. It's not trivial, but it's realistic with the right stack, especially for the mobile/scavenger use-case you're building toward.

Feasibility Breakdown

Text messaging

Very doable.

Low bandwidth, store-and-forward friendly.

Works well even on sparse or intermittent mesh (Bluetooth LE, WiFi Direct, LoRa, or custom UDP overlays).

Existing patterns: Briar, Session, Meshtastic, or simple MQTT-over-mesh.

Voice calls

Feasible but constrained.

- Short-range / dense mesh (WiFi Direct or ad-hoc WiFi): real-time VoIP is practical.
- Longer-range / sparse mesh (LoRa, Bluetooth mesh): delayed or push-to-talk style works better than full-duplex.
  Codecs like Opus at low bitrates (8–24 kbps) keep it lightweight.
  Latency and packet loss are the main enemies; you'd need adaptive buffering and graceful degradation.

Video chat

Hardest of the three.

High bandwidth + low latency required.

Possible only in short-range, high-density mesh with good radios (WiFi Direct or 5 GHz ad-hoc).

On sparse or low-power links it becomes slideshow-quality or store-and-forward video messages rather than live calls.

Practical target: low-resolution (240p–480p), low-framerate, heavily compressed streams, or "video notes" instead of continuous calls.

How to Wire It into HiveNode Containers

1. **Networking Layer**
   - Discovery & routing: libp2p, Yggdrasil, or a lightweight custom mesh (gossip + DHT).
   - Transport: UDP preferred for real-time media; TCP for reliable text.
   - Identity: use the existing ENDCAP + AGP token model so every node stays sovereign and self-describing.
2. **Media Stack**
   - Text: simple encrypted pub/sub or direct peer messaging.
   - Voice: Opus + WebRTC-style media negotiation (or a lighter custom RTP-like protocol).
   - Video: same media pipeline, but with aggressive bitrate control and optional "record-and-forward" fallback.
3. **Container Integration (Termux / PRoot)**
   - Run the mesh daemon and media services inside the container.
   - Expose a local API (or simple CLI/UI) so the HiveNode can start/join calls without root.
   - Battery-aware: Dragon Radar + SRM already monitor power; throttle media quality or switch to text-only when battery is low.
4. **"Void" / Offline Continuity**
   - Store-and-forward queues so messages and short voice/video notes survive when peers are temporarily out of range.
   - Multi-hop relay so a message can travel through intermediate nodes.
   - Local encryption + ENDCAP ownership so data stays under the user's control even while transiting the mesh.

Realistic Targets for a First Version

- Reliable encrypted text + presence.
- Push-to-talk or short full-duplex voice in dense mesh.
- Optional low-res video or video notes when link quality allows.

- Automatic fallback to text when bandwidth or battery drops.
- Works on Android tablets/phones running Termux without requiring root or constant cellular.

Challenges to Design For

- Battery drain from continuous radio scanning.
- NAT-less / pure peer addressing (no public IPs).
- Interference and range limits outdoors (river/woods scenario).
- Security: every media stream must respect AGP tokens and ENDCAP ownership.

This fits the HiveNet pattern perfectly: independent nodes that can still selectively connect and pass real-time or delayed media through a shared mesh layer while remaining complete and sovereign when isolated.

Want to start building?
I can sketch:

- A concrete protocol outline (discovery + media negotiation).
- Termux-friendly stack recommendations.
- Minimal prototype architecture for text + voice first.

Just say which piece you want next (text-first, voice codec choices, container networking, etc.) and we'll move.